

Mal-ASSF: Malware Analysis using Api's Sequence Semantic Fusion

Project Team

Anas Iqbal 19I-1789

Muhammad Awais 19I-1767

Muhammad Subhan 17I-0315

Session 2021-2025

Supervised by

Mr. Jawad Hassan



Department of Cyber Security

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

Contents

LIST OF FIGURES.....	2
Introduction.....	3
1.1 Problem Statement.....	3
1.2 Scope.....	4
1.3 Modules.....	5
1.3.1 Module 1.....	5
1.3.2 Module 2.....	5
1.4 User Classes and Characteristics.....	6
Project Requirements.....	9
2.1 Use-case/Event Response Table/Storyboarding.....	9
2.2 Functional Requirements.....	10
2.2.1 Module 1.....	10
2.2.2 Module 2.....	10
2.3 Non-Functional Requirements.....	10
2.3.1 Reliability.....	10
2.3.2 Usability.....	10
2.3.3 Performance.....	11
2.3.4 Security.....	11
2.3.5 Scalability.....	11
2.4 Domain Model.....	11
Key Entities:.....	11
Relationships:.....	12
System Overview.....	15
3.1 Architectural Design.....	15
1. Presentation Layer (User Interaction).....	15
2. Business Logic Layer (Core Processing).....	15
3. Data Layer (Data Management).....	16
Detailed Flow of the Architecture.....	17
Architecture Diagram Mapping.....	17
3.2 Design Models.....	18
3.2.1 Activity Diagram.....	18
3.2.2 Data Flow Diagram:.....	20
Level 2: Further Breakdown.....	21
3.2.3 System-level Sequence Diagram:.....	22
Participants:.....	22
Interaction Flow:.....	23
3.2.4 State Transition Diagram.....	23
Key States:.....	23
Event Handling and Backend Processes:.....	24
3.3 Data Design.....	25

1. Data Storage and Processing Overview.....	25
2. Entities and Data Structures.....	25
3. Databases and Storage Solutions.....	26
4. Data Flow and Interactions.....	27
References	11

LIST OF FIGURES

Figure 1:Domain Model	10
Figure 2:Architecture Diagram	15
Figure 3:Activity Diagram	17
Figure 4:Data Flow Diagram	19
Figure 5:System Level Sequence Diagram	20
Figure 6:State Transition Diagram	21

Chapter 1

Introduction

This document specifies the requirements for **Malware Analysis Using API Sequence Semantic Fusion**, a dynamic malware analysis tool aimed at detecting malicious binaries by leveraging API sequence data and machine learning techniques. The tool is designed to assist cybersecurity analysts and researchers in identifying malicious behavior in executable files through a combination of dynamic analysis and API2Vec-based semantic fusion.

The intended audience for this document includes:

Developers: Responsible for building and maintaining the system, including its core features such as dynamic malware analysis, API sequence extraction, and machine learning model integration.

Project Managers: Overseeing the project's progress, ensuring that milestones are met, and that the project stays within scope, timeline, and budget.

Users: Primarily cybersecurity analysts and researchers who will interact with the system to submit files for analysis, view reports, and make decisions based on the results.

Testers: Validating the functionality of the system by ensuring that all requirements are met and that the system performs reliably and securely.

Documentation Writers: Responsible for creating user guides, developer documentation, and other resources that support the use and maintenance of the system.

This document will outline the problem being addressed, the scope of the project, the various modules included, and the user classes that will interact with the system.

1.1 Problem Statement

In general, the complexity of the malware as such is gradually growing, and that explains why classic approaches, including static analysis, might often fail to address new threats. Attackers commonly employ technologies such as obfuscation, polymorphism, and packing techniques to triumph over static detection techniques in which one cannot try to identify a malicious code by its look. Furthermore, other current threats include the APTs and a file-less malware that only executes in the memory, and hence require better detection methods. The majority of the available dynamic analysis tools have several issues associated with scalability and accuracy and, most importantly, with understanding when one call follows another in a series of API calls, which is critical in studying the behavior of malware at the time of execution.

While attempting to address these challenges the best strategy for the implementation of this project is the multi-tier malware analysis and detection framework involving dynamic analysis, API sequence analysis and semantic fusion. This way, the system's behavior will be analyzed as to certain binary executables in contained environments and have the API calls of purposely chosen binaries be recoverable and analyzed via a machine learning algorithm to determine malicious actions to provide a much more effective way of detecting malware.

1.2 Scope

The proposed project will seek to establish a desktop application that shall be used to analyze and detect the so called 'malware' or malicious binary executables. The first and foremost goal of this project is to design a solid and effective anti-virus system which may be employed by experts and organizations for the protection of their networks. The application will use Mal-ASSF framework which is based on three stages that involve sequence feature extraction, semantic analysis and machine learning for classification of application into either of the two: either a benign one or a malicious one.

Dataset: To build and to test the model, we will use a various set of samples sourced from reliable resources including MalwareBazaar, Cuckoo Sandbox or VirusTotal, Malwr or the datasets which we will gain from the Microsoft Malware Classification Challenge. This set will contain various types of malware, as well as a number of benign files which will make its basis quite vast for training and testing of the system.

Dynamic Analysis of Executables: The application will start executable files in a secure manner in order to record high level API call sequences and operation.

API Sequence Processing: To be applied to API sequences extracted later, there will be applying some steps like API2Vec that converts API calls into vector representations while preserving semantic associations.

Semantic Feature Extraction: Further improves the detection accuracy by analyzing the semantic relations in API sequences such as API2Vec.

BiLSTM-based Feature Extraction: The application will involve the use of BiLSTM (Bidirectional Long Short-Term Memory) networks in the determination of API sequences while identifying calling relationships and sequential characteristics that are associated with malware.

Attention Mechanism: This will be anticipated to be achieved in sections of API sequences of code by the application of an attention mechanism that will help in capturing behaviors related to malware and thus improving the detection accuracy of the model.

Machine Learning Classification: An MLP classifier will be addressed for the classification of the analyzed executables; dropout will be also applied for the anticipation of overfitting.

Comprehensive Report Generation: Create reports with specific descriptions of the behaviors as well as possible threats found in the analyzed executives.

User Interface: The graphical front end is a stand-alone application that would primarily compile and analyze executable files that are uploaded by users without a lot of complications.

The components mentioned earlier, will be implemented in a way that the project will produce a strong tool for identifying and analyzing the malware hence improving on the security measures, and hence protecting key network resources.

1.3 Modules

1.3.1 Module 1

1. **Data Collection:**

- To gather and prepare a dataset containing malware and benign samples, ensuring safe execution and accurate recording of system interactions.
- Sources include MalwareBazaar or VirusTotal. Tasks include data cleaning, tagging, and preliminary pre-processing.

2. **Sandbox:**

- To securely execute and monitor binary files in an isolated environment, tracking their behavior and interactions with the system.
- Includes virtual environments, execution management for multiple runs, and logging of system activity.

1.3.2 Module 2

1. **API Sequence Extraction & Semantic Analysis Module:**

- Captures and records API call sequences from executing binary files, ensuring timing context is preserved. It also conducts semantic analysis of these sequences using API2Vec and BiLSTM to identify patterns and relationships.
- Includes an API call interceptor, sequence logger, API2Vec for vectorizing API calls, BiLSTM for sequence pattern analysis, and an attention model to focus on relevant sequence parts.

2. **Classification Module:**

- Trains machine learning models to classify API sequences as either malicious or benign, ensuring accurate malware detection.
- Features a Multi-Layer Perceptron classifier, feature engineering tools, and performance analysis mechanisms to enhance model accuracy.

1.4 User Classes and Characteristics

User class	Description
Security Analysts	Security professionals responsible for monitoring and analyzing the malware detection results. They will use the system to identify and investigate potential threats. Analysts will require advanced features for detailed API sequence analysis and report generation.
User	The User will upload malware or benign files to the platform and receive analysis reports generated from sandbox execution and API sequence analysis. This class requires basic knowledge of file analysis processes.
System Administrators	Administrators managing the deployment and upkeep of the application. They will ensure the sandbox environment and classifiers are functioning optimally. They also handle user permissions and system maintenance.
Developers	Software developers working on improving the malware detection system. They will need access to the sandbox environment, API extraction modules, and machine learning models for testing, debugging, and refining system components.

Table 1: User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

The Project primarily involves backend operations like dataset processing, sandboxing, API extraction, and classification, with only a few interactions through a GUI (such as file uploading and viewing reports), an **Event-Response Table** would be a suitable representation.

	Event	System Response
1	User uploads a binary file (malware/benign).	System validates the file, passes it to the Sandbox Module for execution and monitoring.
2	File execution starts in the Sandbox.	System logs interactions, monitors API calls, and extracts the sequence of API calls.
3	API sequence extracted from the file.	The sequence is processed by the API Sequence Extraction Module, preserving timing and context, and passed to the Semantic Analysis Module.
4	API sequence passed for analysis.	The Semantic Analysis Module uses API2Vec and BiLSTM to evaluate the context and relationships between API calls, generating semantic vectors.
5	Semantic vectors ready for classification.	The Classification Module takes the processed API sequence and classifies the file as malicious or benign using machine learning models.
6	Classification completed.	The system generates the analysis result (malicious/benign), preparing it for report generation.
7	User requests to view analysis report.	The system retrieves the classification result and displays the analysis report to the user via the GUI.
8	System generates and exports the analysis report.	The system generates a comprehensive report containing the details of the file's behavior, API sequences, and classification results.

Table 2: Event Response Table

2.2 Functional Requirements

2.2.1 Module 1

Following are the requirements for module 1:

Data Collection & Sandbox Module

1. **FR1:** The system shall collect datasets containing malware and benign samples from sources like MalwareBazaar.
2. **FR2:** The system shall perform data cleaning, tagging, and preliminary pre-processing of collected samples.
3. **FR3:** The system shall execute binary files in an isolated sandbox environment to track system behavior and prevent potential harm.

2.2.2 Module 2

API Sequence Extraction, Semantic Analysis, and Classification Module

Following are the requirements for Module 2:

1. **FR1:** The system shall capture and record API call sequences produced by the execution of binary files.
2. **FR2:** The system shall preserve the timing context of API calls to ensure accurate sequence analysis.
3. **FR3:** The system shall employ API2Vec to convert API calls into vector forms for semantic evaluation.
4. **FR4:** The system shall use BiLSTM to identify patterns in API sequences and an attention model to highlight relevant portions of the sequences.
5. **FR5:** The system shall classify API sequences as malicious or benign using machine learning techniques, including a Multi-Layer Perceptron classifier.

2.3 Non-Functional Requirements

2.3.1 Reliability

- The system shall have a Mean Time Between Failures (MTBF) of 100 hours during normal operation.
- The system shall ensure 99.9% uptime for the malware analysis functionality to support continuous operations for users analyzing files.

2.3.2 Usability

- The system interface shall allow users (including students, security analysts, and researchers) to upload and analyze files with a maximum of 3 interactions (clicks).

- The system shall provide clear feedback and error messages to guide users through the file upload and report generation processes.

2.3.3 Performance

- The system shall process and analyze binary executables up to 10MB in size within 5 minutes using the combined API sequence extraction and machine learning classification methods.
- The system shall support parallel execution of analysis tasks, allowing multiple users to analyze files concurrently without performance degradation.

2.3.4 Security

- The system shall implement Role-Based Access Control (RBAC) to prevent unauthorized access to sensitive components, including analysis reports.
- Files uploaded for analysis shall be quarantined and securely isolated from the rest of the system to prevent harm to other system components.
- Data transmission, including file uploads and report access, shall be encrypted using SSL/TLS to ensure confidentiality and integrity.
- The system shall log all user activities, including file uploads and report generation, for audit trail purposes.

2.3.5 Scalability

- The system shall be able to handle a scalable number of simultaneous file analysis requests, accommodating up to 100 concurrent users without performance bottlenecks.
- The infrastructure shall support the expansion to handle larger datasets, more API sequence analysis, and additional machine learning models as needed.

2.4 Domain Model

To represent the Domain Model, we will focus on the main entities and relationships involved in the system, mapping out the components that interact with each other within the domain of malware analysis.

Key Entities:

1. **User:** The entity that interacts with the system by uploading binaries and viewing analysis reports.
2. **Binary File:** Represents the uploaded executable file (malicious or benign).
3. **Dataset:** A collection of binaries (malware and benign) used for training and testing.
4. **Sandbox:** The environment where binaries are executed in isolation.
5. **API Sequence:** Logs of API calls made by the binary during its execution in the sandbox.

6. **API2Vec**: A tool that converts API sequences into vector representations for analysis.
7. **BiLSTM**: A machine learning model used for analyzing API sequences and extracting patterns.
8. **Classifier**: The entity that classifies the binary based on its API sequence (malicious/benign).
9. **Report**: The final analysis generated for a binary, detailing its behavior and classification.
10. **System**: The application that manages the entire process, from binary upload to report generation.

Relationships:

- User uploads Binary File.
- Binary File is stored in the Dataset.
- Dataset is used by the Sandbox for execution.
- Sandbox produces API Sequences from the binary's behavior.
- API Sequence is processed by API2Vec for vectorization.
- API2Vec passes vectors to BiLSTM for analysis.
- BiLSTM sends the analyzed vectors to the Classifier.
- Classifier produces results, which are included in the Report.
- Report is viewed by the User through the System interface.

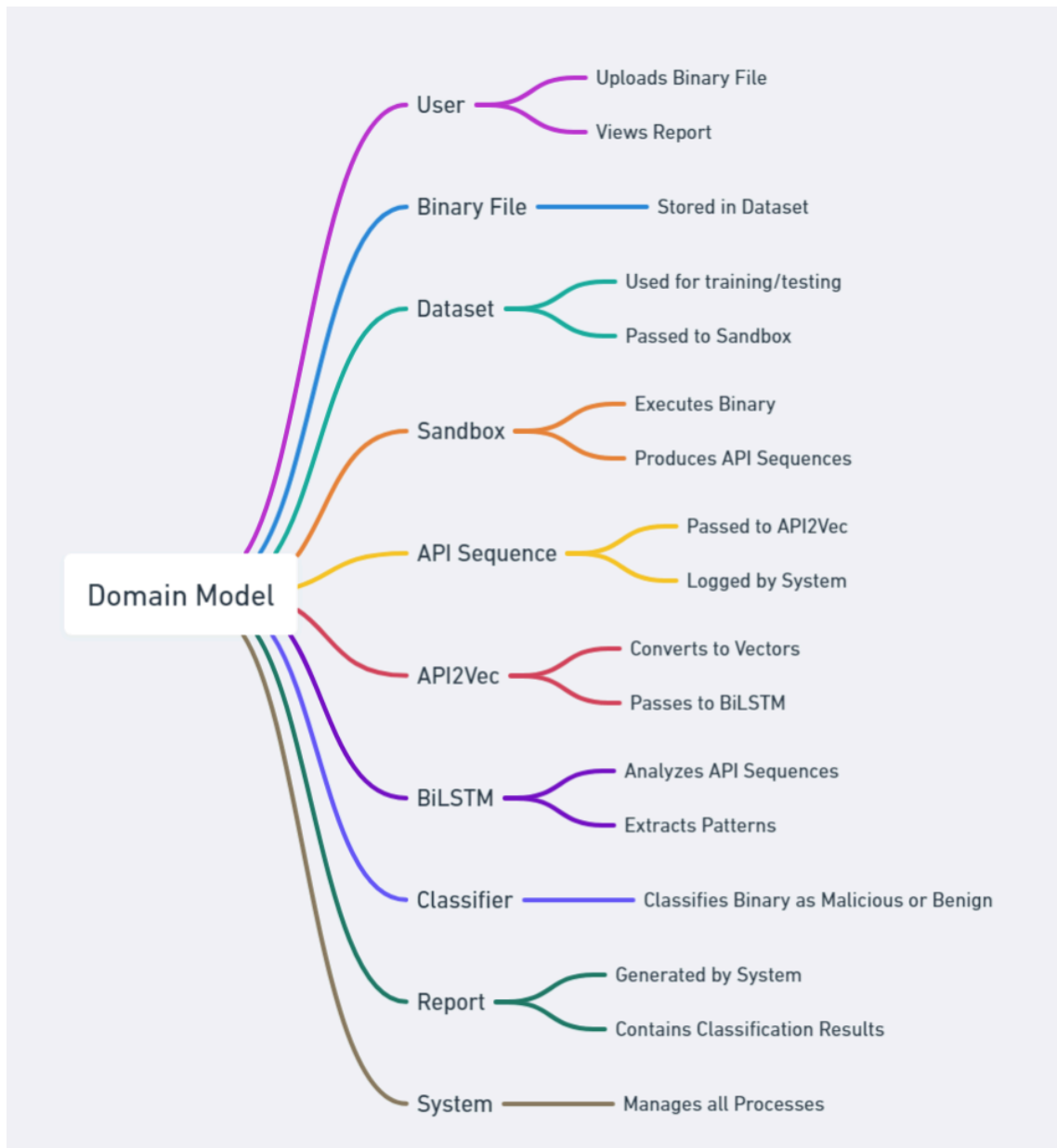


Figure 1:Domain Model

Chapter 3

System Overview

3.1 Architectural Design

we can map each module and its components to a layered architecture. The architecture style is **Layered**, where the system is divided into multiple tiers, and each layer is responsible for specific tasks. This architecture ensures separation of concerns and scalability.

1. Presentation Layer (User Interaction)

- **User Interface Module**
 - File Upload Interface: Allows users to upload files (malware/benign).
 - Analysis Panel: Provides status updates on the file analysis process.
 - Report Viewing Section: Displays the final analysis results (malicious/benign classification) and detailed reports.
 - **Purpose:** Acts as the front-end of the system, allowing users to interact with the backend, submit files, and view reports.
-

2. Business Logic Layer (Core Processing)

- **Sandbox Module**
 - **Components:**
 - Virtual Environment: Isolates the binary execution from the host system.
 - Execution Tracker: Logs API calls, system behaviors, and interactions during the execution.
 - Trace Management: Manages logs for multiple runs and saves trace files for further analysis.
 - **Purpose:** Provides a secure environment for running and observing binaries.
- **API Sequence Extraction Module**
 - **Components:**

- API Interceptor: Captures API calls made by binaries during execution.
 - Sequence Logger: Records API calls and maintains the order and timing context.
 - Preparation Functions: Prepares the captured sequences for further analysis by ensuring data cleanliness and integrity.
- **Purpose:** Captures and prepares API sequences for semantic analysis.
- **Semantic Analysis Module**
 - **Components:**
 - API2Vec: Converts API calls into vector representations to preserve semantic relationships.
 - BiLSTM Model: Analyzes sequential patterns and relationships between API calls.
 - Attention Mechanism: Focuses on critical portions of the API sequences to enhance detection accuracy.
 - **Purpose:** Provides deep analysis of the API sequences to understand the relationship between calls.
- **Classification Module**
 - **Components:**
 - Multi-Layer Perceptron (MLP): A neural network used to classify sequences as malicious or benign.
 - Performance Analysis: Evaluates the model's accuracy and updates parameters to improve results.
 - Feature Engineering: Extracts relevant features from the API sequence data to improve classification accuracy.
 - **Purpose:** Determines whether the binary is malicious or benign based on extracted API sequence data.

3. Data Layer (Data Management)

- **Dataset Gathering and Pre-processing Module**
 - **Components:**
 - Dataset Sources: Integrates with external sources (e.g., MalwareBazaar, VirusTotal) to gather malware/benign samples.
 - Data Pre-processor: Cleans, tags, and prepares datasets for sandbox execution.
 - Dataset Storage: Stores processed datasets for analysis.
 - **Purpose:** Gathers and pre-processes malware and benign files for execution and analysis.
- **Report Generation Module**
 - **Components:**

- **Report Formatter:** Formats the results from the classification and analysis into readable reports.
- **Export Tools:** Allows reports to be exported in different formats (PDF, HTML).
- **Database Connector:** Stores the reports in a database for future reference or auditing.
- **Purpose:** Generates detailed reports on the binary analysis and stores them for later access by users.

Detailed Flow of the Architecture

1. **User Uploads File (Presentation Layer):** The user uploads a binary file via the User Interface Module.
2. **Execution in Sandbox (Business Logic Layer):** The file is executed in the Sandbox Module, where API interactions are logged.
3. **API Sequence Extraction (Business Logic Layer):** The API Sequence Extraction Module captures the API calls from the executed binary and cleans the sequence.
4. **Semantic Analysis (Business Logic Layer):** The cleaned sequence is passed to the Semantic Analysis Module for further analysis using API2Vec and BiLSTM models.
5. **Classification (Business Logic Layer):** The Classification Module uses the analyzed sequence to determine if the binary is malicious or benign.
6. **Report Generation (Data Layer):** A detailed report is created by the Report Generation Module, which summarizes the analysis.
7. **Report Display (Presentation Layer):** The user views the analysis report through the User Interface Module.

Architecture Diagram Mapping

- The Presentation Layer contains the User Interface.
- The Business Logic Layer contains the Sandbox, API Sequence Extraction, Semantic Analysis, and Classification modules.
- The Data Layer contains the Dataset Gathering and Report Generation modules.

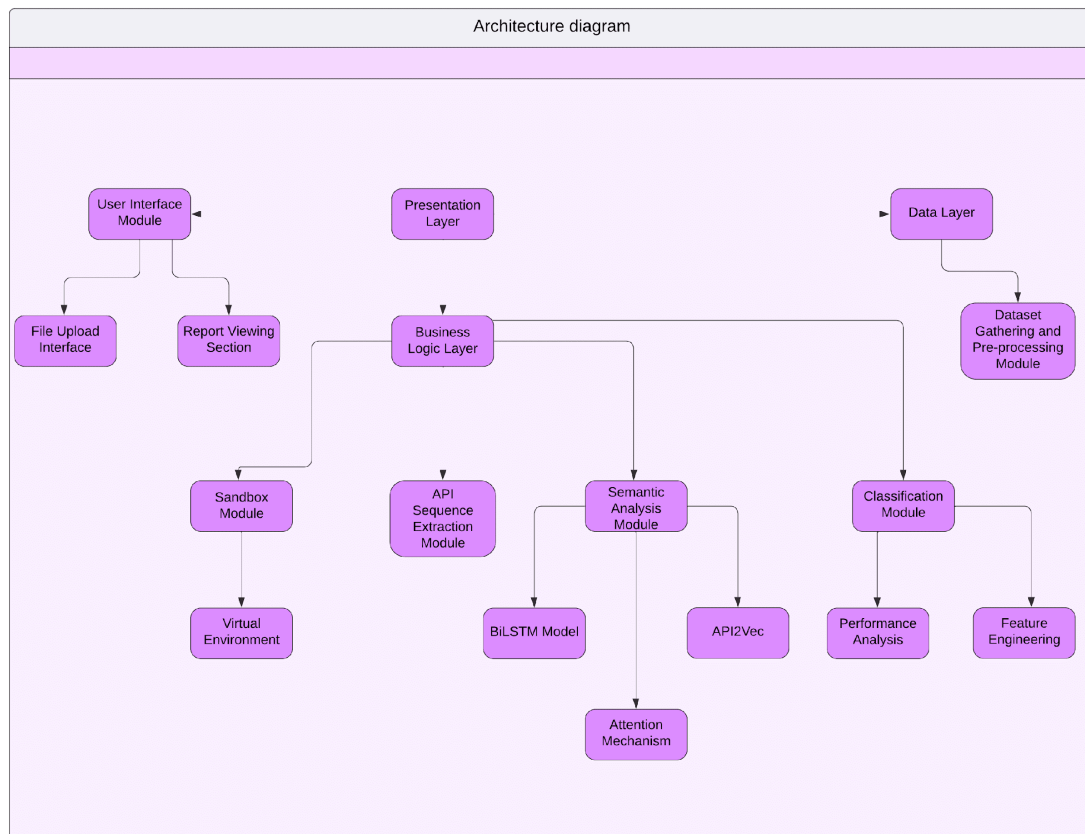


Figure 2:Architecture Diagram

3.2 Design Models

Create design models as are applicable to your system. Provide detailed descriptions with each of the models that you add. Also ensure visibility of all diagrams.

Design Models for Procedural Approach

The applicable models for the project using procedural approach may include:

3.2.1 Activity Diagram

1. Data Set Gathering and Pre-processing:

- This initial step involves gathering the dataset that contains both malware and benign samples.
- Pre-processing tasks include cleaning the data, labeling the samples as malicious or benign, and preparing the dataset for further analysis.

2. Sandbox Execution:

- The system executes the binary files in a controlled, isolated sandbox environment.

- The sandbox tracks the behavior of the binary samples and logs any interactions with the system during execution.

3. API Sequence Extraction:

- This step captures the sequences of API calls made by the binaries during execution.
- The extracted API calls are stored for further analysis, preserving the timing and order of events.

4. Semantic Analysis with API2Vec & BiLSTM:

- The captured API sequences are analyzed using API2Vec (to vectorize API calls) and BiLSTM (Bidirectional Long Short-Term Memory network).
- The purpose of this step is to detect patterns and relationships in the API calls, using deep learning techniques.

5. Decision Point:

- The system evaluates the results from the semantic analysis to classify the binary file as either malware or benign.

6. Malware/Benign Classification:

- Based on the analysis, the binary is classified as either malware or benign.

7. Report Generation:

- Once the binary is classified, the system generates a detailed analysis report summarizing the behavior and classification result (malware or benign).
- The report is presented to the user for further action.

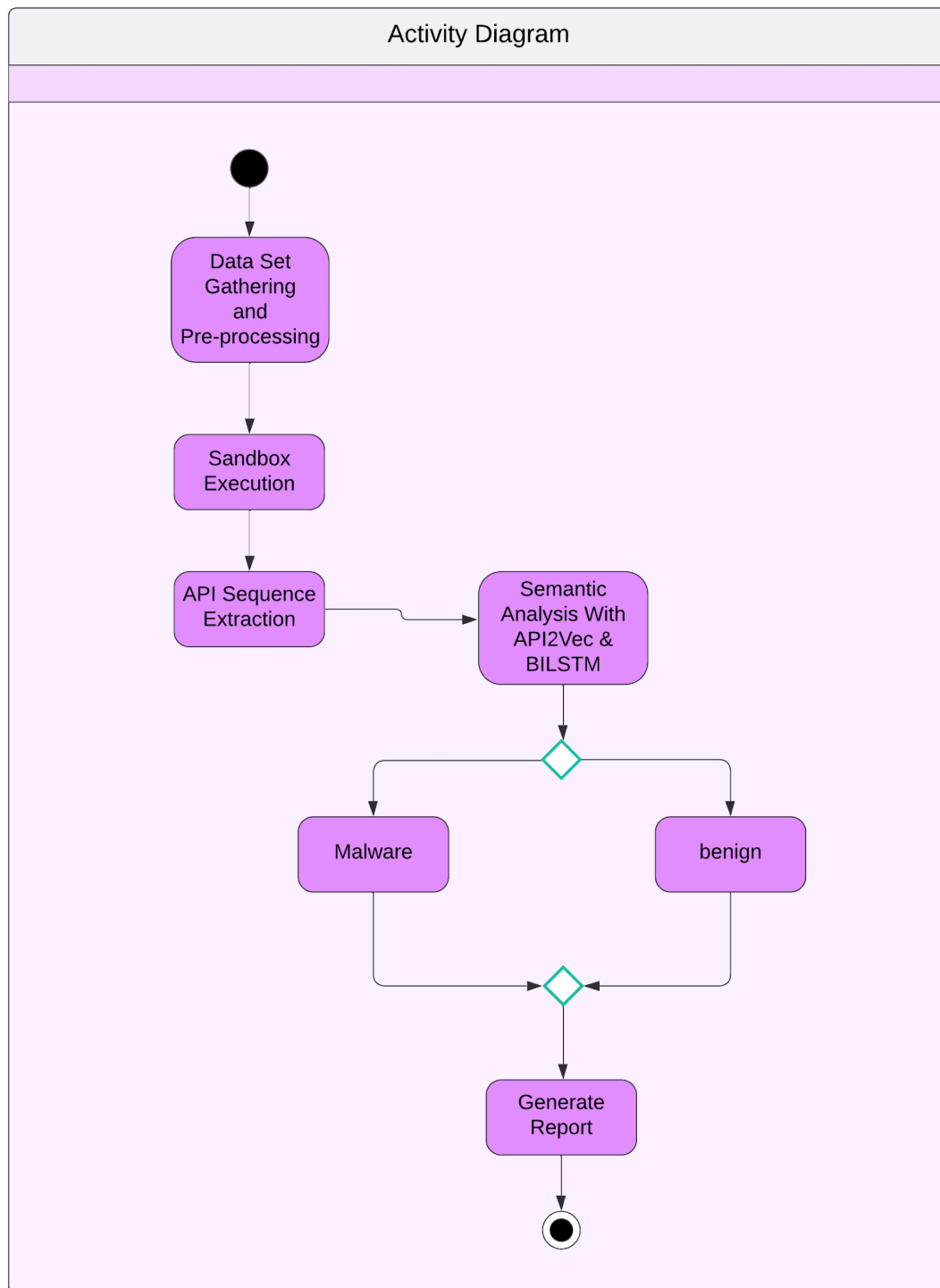


Figure 3:Activity Diagram

3.2.2 Data Flow Diagram:

- Level 1: Breakdown of Core Processes

This will show more detailed sub processes under each main module:

1. **Dataset Gathering & Pre-processing:**
 - Data Collection from external sources
 - Data Pre-processing
2. **Sandbox Execution:**
 - Isolating binary execution
 - Logging API interactions
3. **API Sequence Extraction:**
 - Capturing API sequences
 - Cleaning sequences and maintaining timing
4. **Semantic Analysis:**
 - API2Vec for vector representation
 - BiLSTM for sequence analysis
5. **Classification:**
 - Running machine learning models (MLP)
 - Classifying API sequences
6. **Report Generation:**
 - Compiling detailed analysis reports
7. **User Interface Interaction:**
 - Uploading binaries
 - Viewing reports

Level 2: Further Breakdown

At this level, we can detail the specific processes within each module:

- Data Collection: Fetching malware and benign binaries from sources like VirusTotal, MalwareBazaar, etc.
- Data Pre-processing: Cleaning, tagging, and structuring the dataset for analysis.
- API Interceptor: A specific subprocess to intercept API calls in the Sandbox.
- Vectorization: API2Vec to convert sequences into vectors for analysis.
- Attention Mechanism: Applying attention models to focus on specific parts of the API sequence.

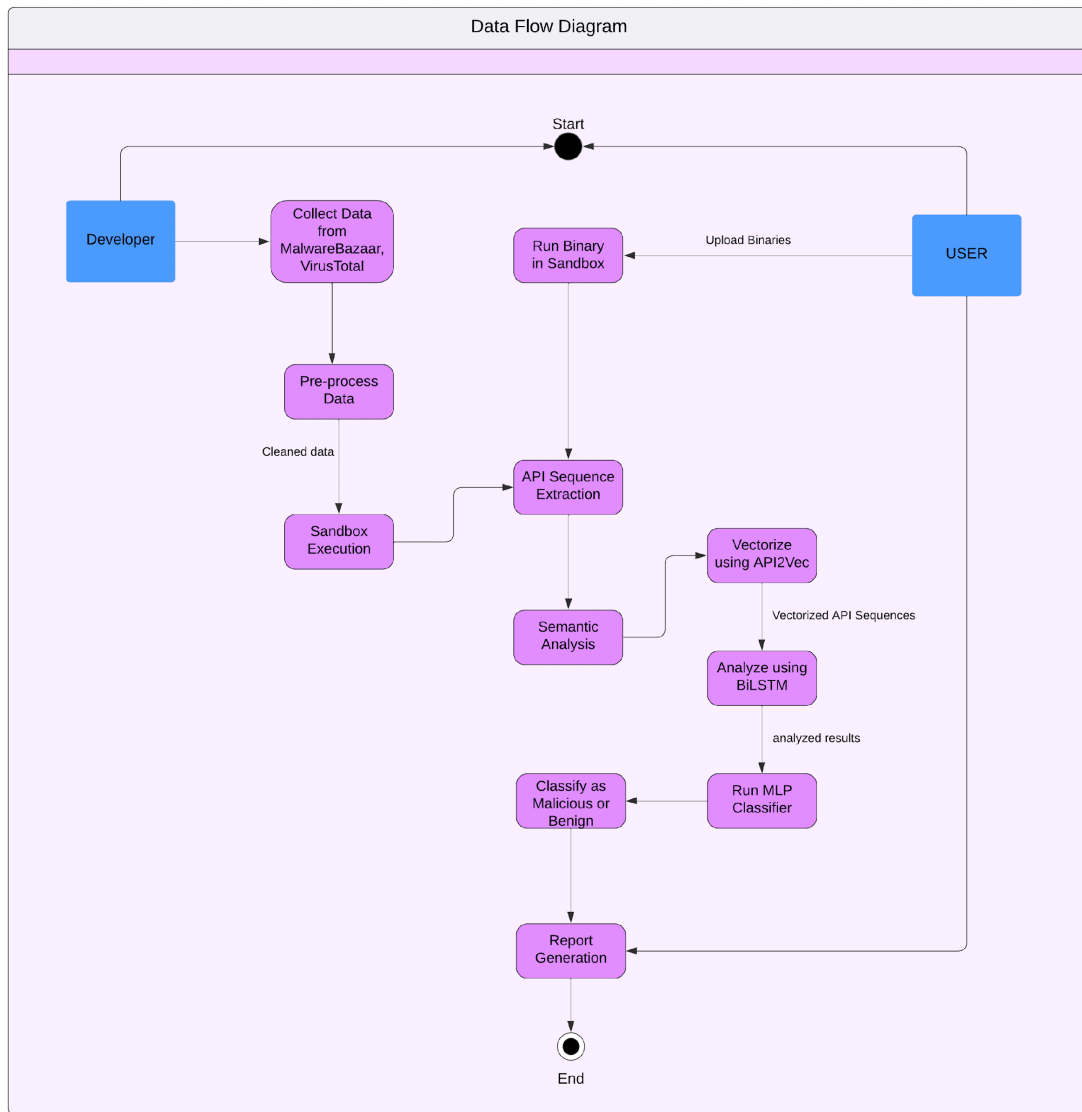


Figure 4:Data Flow Diagram

3.2.3 System-level Sequence Diagram:

Participants:

1. User: Initiates the malware analysis by uploading binaries and views the results.
2. Dataset Gathering Module: Collects and pre-processes binaries.
3. Sandbox Module: Runs binaries in a controlled environment.
4. API Sequence Extraction Module: Extracts API sequences from the executed binaries.
5. Semantic Analysis Module: Processes API sequences for semantic analysis.
6. Classification Module: Classifies the binaries as malicious or benign.
7. Report Generation Module: Generates detailed reports of the analysis.
8. User Interface: Displays the final reports to the user.

Interaction Flow:

1. User uploads a binary file via the User Interface.
2. Dataset Gathering Module fetches or processes the binary.
3. Sandbox Module runs the binary and logs API interactions.
4. API Sequence Extraction Module captures and cleans API sequences.
5. Semantic Analysis Module performs vectorization and analysis on API sequences.
6. Classification Module classifies the file as malicious or benign.
7. Report Generation Module compiles the results and prepares a report.
8. User Interface displays the generated report to the user.



Figure 5: System Level Sequence Diagram

3.2.4 State Transition Diagram

The **State Transition Diagram** for the system will show how the system transitions between different states based on events and the associated backend processes.

Key States:

1. Idle: The system is waiting for user input.
2. Uploading File: The user uploads a binary for analysis.

3. Sandbox Execution: The system runs the binary in a controlled environment.
4. Extracting API Sequences: The system captures and cleans the API sequences.
5. Semantic Analysis: API sequences are analyzed using API2Vec and BiLSTM.
6. Classification: The system classifies the binary as malicious or benign.
7. Report Generation: The system generates a detailed report of the analysis.
8. Report Display: The generated report is displayed to the user.

Event Handling and Backend Processes:

- File Upload: Triggers transition from Idle to Uploading File.
- Sandbox Execution: Starts after preprocessing is complete, moving to Sandbox Execution state.
- API Sequence Extraction: Starts once the binary has been executed.
- Semantic Analysis: Begins after API sequences are cleaned and captured.
- Classification: The system classifies the API sequences.
- Report Generation: Generates the final analysis report.
- Report Display: Displays the generated report to the user

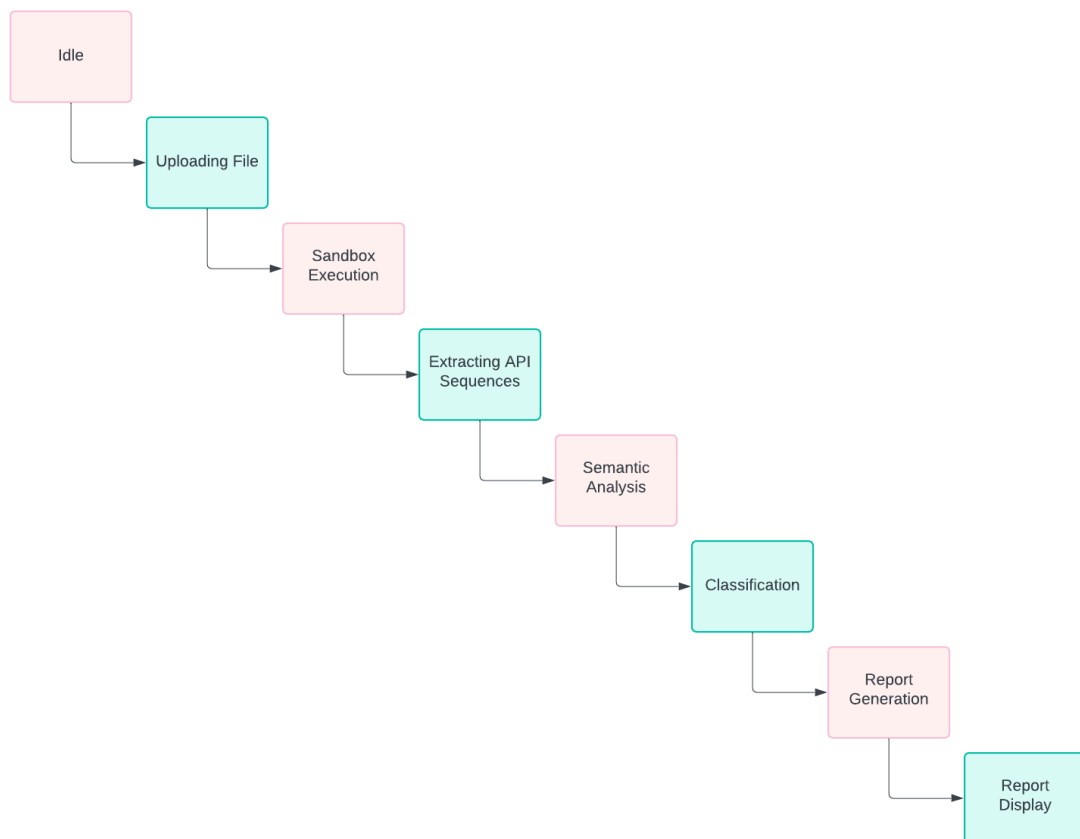


Figure 6: State Transition Diagram

3.3 Data Design

the information domain is transformed into well-structured data entities to facilitate smooth storage, processing, and analysis of binary files for malware detection. The major system entities, along with their data storage mechanisms, are described below:

1. Data Storage and Processing Overview

The system primarily deals with:

- Binary files (malicious and benign).
- API sequences generated during file execution.
- Semantic representations of these API sequences.
- Analysis reports generated after classification.
- User data for access and interaction with the system.

The following subsections explain how each of these entities is stored, processed, and organized.

2. Entities and Data Structures

1. Binary Files

- **Storage:**
 - Files uploaded by users are stored in a secure, isolated environment (sandbox) for further analysis.
 - Files are stored temporarily during execution and removed after analysis.
- **Processing:**
 - Files are executed in a sandbox environment where API call sequences are captured.
- **Data Organization:**
 - Files are classified as benign or malicious after processing.
 - Metadata for files (such as size, format, and hash values) is maintained for tracking and validation.

2. API Sequences

- **Storage:**
 - API call sequences are stored in structured log files or a database, maintaining the order of execution, timing, and parameters passed.
- **Processing:**
 - Sequences are processed for feature extraction using the API2Vec model and further analyzed using the BiLSTM model to capture sequential dependencies.
- **Data Organization:**
 - API calls are organized by unique identifiers for the file being analyzed, and include timestamps, API names, and parameters.

3. Semantic Representations (API2Vec Embeddings)

- **Storage:**
 - Semantic vectors generated by the API2Vec model are stored in the system's memory or database for further processing by the classification module.
- **Processing:**
 - These vectors represent API sequences in a high-dimensional space, which is then fed into machine learning models for classification.
- **Data Organization:**
 - Vector representations are stored with corresponding API call sequence IDs for traceability and reference in the classification process.

4. Analysis Reports

- **Storage:**
 - Reports are stored in a database and associated with the corresponding file and user.
 - Report components such as file classification (benign/malicious), API sequences, and potential threats are stored.
- **Processing:**
 - Reports are generated after the classification process and are made available for user viewing and downloading.
- **Data Organization:**
 - Reports are indexed by file, user, and date of analysis, ensuring easy retrieval for review or auditing.

5. User Data

- **Storage:**
 - User data, including access credentials and activity logs, is stored in a secure database.
- **Processing:**
 - User roles are processed to determine appropriate access levels using role-based access control (RBAC) mechanisms.
- **Data Organization:**
 - Users are categorized into different roles (e.g., admin, student/user) with relevant permissions stored and enforced by the system.

3. Databases and Storage Solutions

- **Relational Database** (e.g., PostgreSQL or MySQL):
 - This database will store structured data such as:
 - **User credentials and roles** for access control.
 - **Analysis reports** linked to file analysis results.

- **Metadata** related to files and API sequences.
- **NoSQL Database** (e.g., MongoDB):
 - For storing large, unstructured data such as:
 - **API sequences** with their associated parameters.
 - **Semantic vectors** (API2Vec embeddings).
- **Log Files:**
 - API call logs, execution traces, and error logs are stored as text files or log files for auditability and debugging.

4. Data Flow and Interactions

- **User Interface:** Users upload binary files through the interface. The files are stored temporarily and processed for malware analysis.
- **Sandbox Module:** Executes the files and captures API sequences.
- **API Sequence Extraction and Semantic Analysis Modules:** Extract and analyze API sequences, producing vector representations.
- **Classification Module:** Processes the semantic vectors and classifies the files as malicious or benign.
- **Report Generation Module:** Stores the result of the analysis and provides users with detailed reports.

This data design ensures a secure, organized, and efficient transformation of the raw binary files into meaningful analysis, facilitating the detection of malware with robust backend processing and data organization.

Bibliography

- Wu J. Zhang M. Yang W. Zhang, S. Dynamic malware analysis based on api sequence semantic fusion. *Journal of Computer Security*, 2023.
- Zhou, Y., & Hu, J. (2021). Effective malware detection using API call sequences with attention mechanisms. *Proceedings of the International Conference on Data Mining*.